

TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems

1st Minxing Wang

Singapore Management University
Singapore, Singapore
mxwang@smu.edu.sg

2nd Xiaofei Xie

Singapore Management University
Singapore, Singapore
xfxie@smu.edu.sg

3rd Yintong Huo

Singapore Management University
Singapore, Singapore
ythuo@smu.edu.sg

Abstract—Agentic systems have been widely studied to automate coding tasks such as bug fixing and feature implementation. As these systems increasingly operate on complex codebases, understanding where and why they fail becomes essential for iterative refinement and operational reliability. Existing automated failure diagnosis approaches leverage *task execution trajectories*, yet they struggle with trajectories produced by repository-level coding agents due to two key properties. First, these trajectories are often long, spanning many execution steps, making it difficult for LLMs to track the causal chain of failure over the execution history. Second, these trajectories are laden with noise, containing substantial low-signal observations such as redundant program structures and verbose code context, which can interfere with LLM reasoning.

To address these challenges, we propose *TrajAudit*, an automated failure diagnosis framework specifically for trajectories produced by repository-level coding agents. TrajAudit employs an investigator agent supported by two modules: one reduces failure-irrelevant noisy context through semantic saliency folding, and the other derives preliminary diagnostic guidance from test failure reports as prior knowledge to help LLMs focus on likely failure regions. The investigator agent can further invoke tools to inspect folded content on demand, enabling a focused investigation without losing access to the full trajectory context. We also introduce *RootSE*, a benchmark of 102 real-world agentic failure instances from repository-level coding tasks, each annotated with the earliest decisive error step and a diagnostic justification. Experiments on RootSE show that TrajAudit outperforms the strongest baselines by 10.8% and 21.6% in exact failure localization accuracy in the with- and without-reference settings, respectively, demonstrating its effectiveness.

Index Terms—Agentic Systems, Failure Diagnosis, Software Maintenance

I. INTRODUCTION

LLM-based agentic systems are autonomous systems powered by large language models (LLMs) that perceive environmental states, perform goal-oriented reasoning, and execute actions in a closed-loop manner [1]–[3]. Recently, a growing number of LLM-based agents have been developed to automate *repository-level coding tasks*, like issue resolution and feature development [4]–[8].

While these agents succeed in simple tasks such as single-file modifications, they still struggle with complex, multi-file tasks that require long-horizon reasoning and execution [9]–[12]. These failures occur in opaque ways, often as the cumulative consequence of a single early mistake, such as a

misunderstanding of the task requirements or a flawed implementation plan [10], [13]. Therefore, understanding where and why agents fail is critical for the iterative refinement of agentic systems and, ultimately, trustworthy intelligent software engineering [13]–[16].

Execution trajectory, which records sequential steps of the agent’s reasoning, tool invocations, and environmental observations, provides the key information to monitor the agent’s behavior [17], [18]. As shown in Figure 1, each step contains four types of information: the thinking process (Thought), the transitional language produced by the agent (Response), the actions taken (Action), and the environmental feedback (Observation). These trajectories are commonly used for understanding failures behind agent execution. For example, one pioneering study conducted in-depth manual investigations of inter-step inconsistencies to characterize failure patterns, such as reasoning-action conflicts [19]. To further automate *failure diagnosis*, existing LLM-based approaches ask LLMs to inspect complete trajectories [20], [21], while spectrum-based approaches identify suspicious steps by contrasting failed and successful runs [22]. However, neither handles long and noisy repository-level coding trajectories well.

We identify two distinct challenges as follows, (1) *Observational noise*. Observations refer to the information returned by tools invoked by the agent, often accounting for over 70% of the total trajectory content [20]. However, most observations are not directly relevant to diagnosis, such as redundant program structures and verbose code context, which would complicate automated failure diagnosis [23]. (2) *Excessive length*. These trajectories often span from 20 to over 100 steps, with each step containing on average over 5,000 characters. Existing studies [24]–[26] show that even state-of-the-art LLMs struggle to maintain reasoning quality when processing long contexts.

Both challenges stem from a fundamental limitation of existing diagnosis methods: they passively consume entire trajectories, treating all steps as equally relevant. Inspired by the action-taking nature of agents [27], [28], we propose an agentic approach that *actively explores and fetches* the most relevant fragments from long and noisy trajectories.

To this end, we introduce **TrajAudit**, an automated failure diagnosis framework for repository-level coding agent trajectories, including pinpointing the first step at which the agent

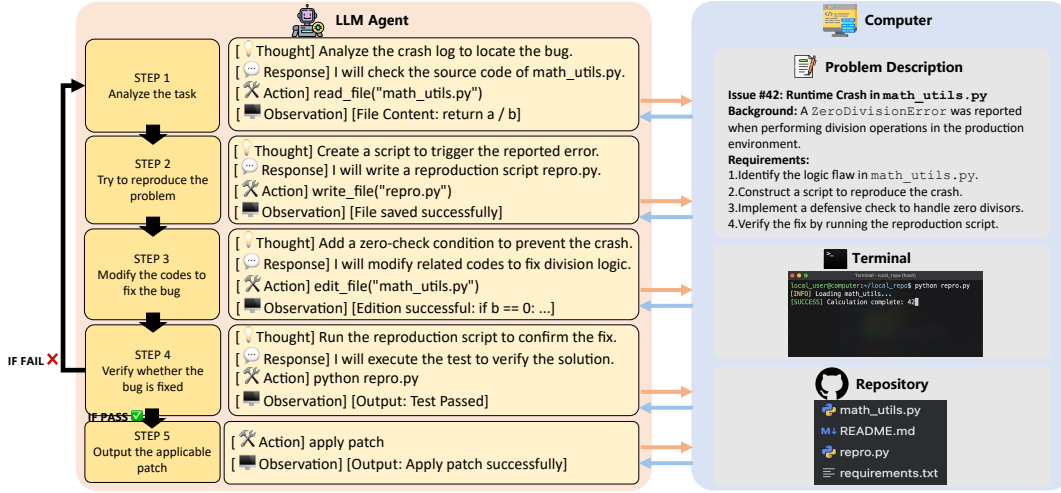


Fig. 1: The agent workflow and execution trajectory in a coding task.

commits a decisive error (i.e., *earliest decisive error step*) and providing diagnosis justifications. TrajAudit addresses the aforementioned challenges through an agent with two synergistic modules. (1) *Prior failure reasoning* derives a preliminary diagnosis by prompting an LLM to identify the most suspicious region responsible for the failure based on the test case errors. This diagnosis is then incorporated into the investigator agent’s context as prior knowledge, directing the model’s focus on failure segments, thereby mitigating long-context degradation. (2) *Semantic saliency folding* selectively compresses trajectory observations by retaining only failure-relevant context, such as code patch structures and entries containing failure indicators (e.g., ‘fail’, ‘exception’). (3) *Investigator agent* enables dynamic access to the full trajectory, performing on-demand retrieval of folded content through predefined interactive tools. This mechanism enables a top-down diagnostic approach, allowing the diagnosis to begin with a high-level overview and selectively drill down into details on demand, thereby mitigating observational noise.

Furthermore, to evaluate TrajAudit, we introduce **RootSE**, the first benchmark to evaluate a model’s ability to diagnose agent execution failure in completing repository-level coding tasks. The dataset consists of 102 instances with over 5,000 execution steps, offering a challenging testbed for failure diagnosis. Experimental results on RootSE show that TrajAudit consistently outperforms all baselines, achieving up to 21.6% improvement in exact step-level accuracy over the strongest baseline, demonstrating its effectiveness for diagnosing coding agents.

To sum up, the main contributions are threefold:

- We introduce TrajAudit, an automated failure diagnosis framework for repository-level coding agent trajectories, which employs an investigator agent supported by semantic saliency folding and prior failure reasoning to localize the earliest decisive error step and generate diagnostic justifications.
- We construct RootSE, the first benchmark for earliest-step

failure diagnosis on repository-level coding task trajectories, consisting of 102 real-world instances annotated with earliest decisive error steps and diagnostic justifications.

- We release all artifacts to facilitate independent verification, reproducibility, and future research.

II. PRELIMINARIES

In this section, we formally define the execution trajectory and the failure diagnosis task, which serve as the foundation for the benchmark and methods in later sections.

A. Execution Trajectory

We model an LLM-based agentic system as a stateful system that interacts with a software environment through a sequence of discrete steps to complete a given coding task. Following prior work [20], we represent a full execution trajectory as:

$$\tau = (s_0, \eta_1, s_1, \eta_2, s_2, \dots, \eta_T, s_T) \quad (1)$$

where s_0 is the initial system state, s_t is the system state after step t , and η_t denotes the agent-environment *interaction step* at time t . A *state* represents the available execution context, including the task description, repository snapshot, accumulated interaction history, and tool outputs. Each *interaction step* is represented by four components: a **Thought** (the reasoning trace explicitly logged by the agent), a **Response** (the transitional natural language statement generated by the agent to describe its current or intended operation), an **Action** (the tool invocation by the agent), and an **Observation** (the environmental feedback returned by the tools). We define a binary outcome function $Z(\tau) \in \{0, 1\}$, where $Z(\tau) = 1$ indicates task failure and $Z(\tau) = 0$ indicates task success. In the context of coding tasks, task success is determined by whether all acceptance test cases pass.

B. Failure Diagnosis

The failure diagnosis task is defined as follows: given a task description, a failed trajectory, the test code, and the corresponding test failure output, identify the first step at which the agent commits a decisive error (i.e., the earliest decisive error step) and provide a natural language justification explaining how that step leads to task failure.

We adopt the definition of the earliest decisive error step from Zhang et al. [20]. Given a failed trajectory τ with $Z(\tau) = 1$, we define η_t as a decisive error step if there exists an alternative interaction $\bar{\eta}_t$ such that replacing η_t with $\bar{\eta}_t$, while keeping all preceding steps unchanged, yields a modified trajectory $\bar{\tau}$ satisfying $Z(\bar{\tau}) = 0$. Among all decisive error steps in τ , the one with the smallest step index is defined as the earliest decisive error step.

III. RELATED WORK

A. Failure Diagnosis Methods

1) *LLM-based*: Zhang et al. [20] proposed three methods that feed trajectories into LLMs through single-pass, sequential, and binary-search strategies to identify the decisive failure step. Deshpande et al. [21] introduced a taxonomy of 20 agentic error types, such as formatting errors and instruction non-compliance, and located all failed steps within a trajectory by prompting an LLM with both the taxonomy and the full trajectory. These methods can be effective for tasks with short execution traces, such as web browsing. However, their effectiveness collapses on coding task trajectories. Such long trajectories may exceed the context window of LLMs, making methods that require the full trajectory impractical. In addition, tool outputs in coding task trajectories often contain substantial noise, such as irrelevant code context and verbose file structures. Such noise interferes with the LLM’s ability to identify the decisive failure step.

2) *Spectrum-based*: Ge et al. [22] proposed FAMAS, which adapts the idea of spectrum-based fault localization to diagnose agentic systems. FAMAS first re-executes the failed task multiple times to gather a set of trajectories. It then normalizes each into standardized behavior triples using an LLM and clusters them into semantically equivalent groups. Finally, it ranks groups by frequency-based suspiciousness score to pinpoint failure indicators.

While FAMAS outperforms prior purely LLM-based methods on general-purpose task failure localization [22], it faces two limitations when applied to coding tasks. First, repeated re-execution is costly, as each run costs nearly one dollar on average [29], making scaling debugging impractical. Second, by representing agent behavior through action-based triples, FAMAS overlooks reasoning traces, which are a key failure signal in coding agents.

3) *Learning-based*: Zhang et al. [18] proposed AgenTracer, which fine-tunes a lightweight model on annotated failure trajectories via reinforcement learning to identify the decisive failure step. However, this approach requires training a dedicated model, making it less flexible to deploy in new settings.

B. Failure Diagnosis Benchmarks for Agentic Systems

Zhang et al. [20] proposed Who&When, the first benchmark for failure diagnosis in agentic systems, comprising 184 annotated trajectories with ground-truth labels for the earliest decisive error step. However, their trajectories came from general-purpose tasks, which tend to be shorter and less noisy than coding ones, limiting the applicability to complex SE settings. Deshpande et al. [21] proposed TRAIL, a benchmark of 148 annotated trajectories, of which 118 are information retrieval tasks and only 30 involve software engineering. Furthermore, TRAIL recognizes all plausible erroneous steps, such as formatting errors and hallucinations. In contrast, our work focuses on identifying the earliest crucial step that leads to the failure.

IV. THE ROOTSE BENCHMARK

Who&When, the most closely related benchmark for earliest step failure diagnosis, consists of trajectories from general-purpose tasks such as web browsing [30]. As shown in Figure 2, trajectories from general-purpose tasks differ from those produced during repository-level coding tasks (e.g., SWE-bench Pro) in two important ways: (1) observation content accounts for a substantially smaller proportion of the trajectory, and (2) the trajectories span far fewer steps [31]. This lower proportion of observation content suggests that reasoning, actions, and their outcomes are less obscured by noisy context such as irrelevant code snippets and repository file structures, reducing the amount of low-signal context that LLMs must process. These differences suggest that Who&When does not fully capture the difficulty of coding tasks, and no existing benchmark evaluates the earliest step failure diagnosis in this setting.

To fill this gap, we introduce RootSE, the first benchmark for earliest step failure diagnosis on repository-level coding task trajectories, characterized by long-horizon execution as well as substantial observation content. It comprises 102 failed instances generated by four representative agents tackling diverse repository-level coding problems, each annotated with the earliest decisive error step and a diagnosis justification. Together, these instances encompass more than 5,000 individual execution steps and approximately 30 million characters, representing a challenging and realistic evaluation setting.

A. Task Description

Each instance in RootSE comprises four core elements: (1) **Task Specification**: encompasses all metadata required for task execution and completion verification, including the task description, repository name, base commit, and test code patch; (2) **System Configuration**: identifies the specific agentic system and the underlying LLM employed; (3) **Failure Context**: consists of the complete execution trajectory and the test error messages; and (4) **Ground-truth Labels**: includes the earliest error step, diagnosis justification, and the reference patch for the task.

In particular, RootSE asks a diagnosis model to take the task description, failure trajectory, test code, and corresponding

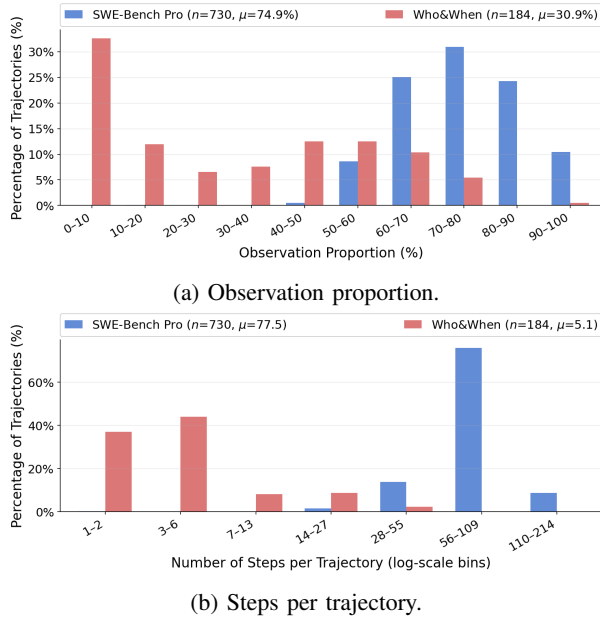


Fig. 2: Comparison of trajectory characteristics between SWE-bench Pro and Who&When.

error messages as input, and output the failure step, along with its justification. RootSE employs three metrics to evaluate the results: **(1) Exact Step-Level Accuracy**, which measures the percentage of instances where the predicted step matches the ground truth exactly; **(2) Tolerated Step-Level Accuracy**, which represents the proportion of predictions that fall within a predefined tolerance window around the ground truth; and **(3) Justification Accuracy**, which measures the percentage of instances where the predicted diagnosis justification is semantically equivalent to the ground truth, as checked by LLM-as-a-judge.

B. Data Collection

1) Agentic System Selection: We select four agentic systems as trajectory sources: SWE-agent, Live-SWE-agent, OpenHands, and AutoCodeRover. These systems are either widely adopted or technically distinctive. SWE-agent introduces the Agent-Computer Interface (ACI), which constrains the action space and standardizes environment feedback to improve the reliability of agent-environment interactions [32]. Live-SWE-agent builds upon SWE-agent with a single prompt modification that enables the agent to create task-specific tools on demand [33]. OpenHands provides a sandboxed runtime environment along with a web-based GUI that supports iterative software maintenance workflows [6]. SWE-agent and OpenHands have each received over 15,000 GitHub stars. AutoCodeRover represents programs as abstract syntax trees, enabling code search at the granularity of classes and methods for efficient task resolution [5], and has been commercialized.

We restrict trajectory sources to single-agent systems for two reasons. First, RootSE focuses on diagnosing failures caused by an agent’s own reasoning and actions, such as

misunderstanding task requirements and incomplete code implementations. Since research indicates that 37% of multi-agent system (MAS) failures stem from inter-agent cooperation breakdowns [17], including MAS trajectories would introduce confounding factors that fall outside the scope of this benchmark. Second, since each agent in a MAS operates on its own individual trajectory, TrajAudit can still be applied to diagnose reasoning and action failures of individual agents within MAS settings.

2) Task Selection: We select task sources based on three criteria: tasks must originate from real-world repository-level issues, support automatic verification through executable test suites, and require multi-step code reasoning and execution. Based on these criteria, we adopt SWE-bench [29], SWE-bench Pro [31], and SWE-rebench [34] as task sources. SWE-bench comprises 2,294 problems from real GitHub issues across 12 Python repositories. SWE-bench Pro targets more challenging cross-file tasks, requiring modifications spanning an average of 4.1 files across multiple programming languages. SWE-rebench is an automatically constructed dataset of over 21,000 tasks designed to provide unseen problems to mitigate the contamination risk.

3) Trajectory Generation: We collected candidate trajectories by running the selected agentic systems on the chosen benchmarks, using representative LLM backbones including ChatGPT, Qwen, Claude, and Gemini series, and by incorporating trajectories from the public dataset [35], yielding an initial pool of 500 candidate trajectories. These trajectories were then filtered through manual inspection in three steps. First, we removed trajectories with ambiguous task descriptions, leaving 412 trajectories. Second, we excluded cases whose test suites checked requirements not mentioned in the task description, leaving 364 trajectories. Third, we retained only cases with exactly one failing test, since trajectories with multiple failing tests may reflect simultaneous deviations from several distinct requirements, making it difficult to identify a single earliest decisive error step, leaving 102 trajectories. The resulting benchmark spans 35 repositories, three programming languages, and four agent architectures; a detailed analysis is provided in Section IV-D2.

C. Annotation Procedure

After collecting data, we annotate each instance with the earliest decisive step index and a corresponding justification. To ensure annotation quality, the procedure involves three stages and three human experts: two annotators (A_1, A_2), each with three years of software development experience and prior experience using coding agents, and one validator (V_0) with extensive experience in developing coding agents, who arbitrates unresolved disagreements between A_1 and A_2 .

Stage I: In the first stage, the three experts independently review the task definition and draft candidate annotation criteria. They then consolidate these criteria into a unified guideline through group discussion, as shown in Figure 3.

Stage II: In the second stage, A_1 and A_2 independently annotate the entire dataset following the unified guideline.

Annotation Guideline

Key Notes:

- a) An error can occur at the reasoning level (flawed planning or incorrect inference) or the execution level (sound reasoning but faulty action, such as improper tool use or faulty code editing).
- b) A step is marked as an error only when the agent has settled on a flawed direction, not when it explores multiple hypotheses, even if some of them are incorrect.

Justification:

Provide an explanation of why this step is the earliest decisive error step and how it leads to the eventual task failure.

Example:

At Step 1, the agent misreads the issue as requiring a new function rather than a bug fix, causing the entire implementation to address the wrong objective and ultimately failing all test cases.

Fig. 3: RootSE annotation guideline.

Each annotation consists of two core elements: the index of the earliest decisive error step and a textual justification explaining how this step causes the system to deviate from the correct solution and ultimately leads to task failure. **Stage III:** In the final stage, A_1 and A_2 discuss all annotation inconsistencies to reach a consensus. For any unresolved disagreements, V_0 first independently annotates the disputed cases and then joins the discussion to reach a final consensus.

To evaluate annotation reliability, we measure the inter-rater agreement between A_1 and A_2 following Stage II. The Cohen’s Kappa coefficient for earliest decisive error step identification is 0.78, indicating substantial agreement [36]–[38]. All remaining discrepancies are subsequently resolved in Stage III through the arbitration of V_0 , producing the final consensus annotations used in RootSE. To further validate the annotations, we conduct a counterfactual verification study on a random sample of 30 instances. For each instance, the annotators construct a corrective replacement for the annotated step without consulting the reference patch, for example, replacing flawed reasoning with a corrected diagnosis or plan, or replacing a faulty code edit with a corrected one, while keeping the preceding trajectory unchanged. The agent is then resumed from the modified trajectory under the same execution setting. All 30 instances result in successful task completion. Although this result does not prove that the annotated step is the unique cause of failure, it provides empirical support that it lies on a causal path to the observed failure.

D. Benchmark Analysis

We analyze RootSE from two perspectives: failure phase distribution and the complexity and diversity of tasks and trajectories.

1) *Failure Phase Distribution Analysis:* Following the standard workflow of automated program repair, which consists of the Localization Phase, Patch Generation Phase, and Verification Phase [39], we use these phases as a coarse-grained abstraction of repository-level code modification. Based on the purpose of the annotated error step in each trajectory, we map each RootSE instance to the corresponding phase or category.

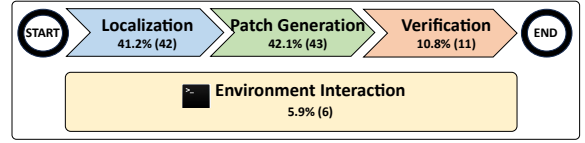


Fig. 4: Phase-wise failure distribution in RootSE.

TABLE I: Comparison of dataset complexity across multiple dimensions for RootSE and Who&When.

Metric	Who&When	RootSE (Ours)
Problem Source	Personal Assistant	Software Maintenance
#Task Desc. Chars	240.5	8,186.7
#Steps	5.2	51.7
#Chars per Step	3,919.0	5,567.8

As shown in Figure 4, failures are not confined to a single phase, with the Localization Phase accounting for 41.2% (42 instances), the Patch Generation Phase for 42.1% (43 instances), and the Verification Phase for 10.8% (11 instances), indicating that RootSE captures failure scenarios across the main activities of locating, modifying, and validating code changes. In addition, we identify environment-interaction failures as an orthogonal category (5.9%, 6 instances), including failures in tool usage or command execution. These failures may arise at different points in the whole coding process.

2) *Complexity and Diversity Analysis:* We evaluate RootSE’s complexity along two dimensions: task complexity and trajectory complexity. Table I compares statistics between RootSE and Who&When dataset.

For task complexity, we use one metric: the average character count of task descriptions (#Task Desc. Chars). As shown in Table I, task descriptions in RootSE average 8,186.7 characters, about 34 times longer than the 240.5 characters in Who&When, reflecting substantially higher contextual complexity.

For trajectory complexity, we adopt two metrics: the number of steps per trajectory (#steps) and the average character count per step (#chars per step). As shown in Table I, on average, RootSE trajectories contain nearly ten times as many steps as Who&When, and each step is also 42% longer. These extended and highly detailed trajectories impose higher requirements on failure diagnosis methods.

Beyond complexity, RootSE exhibits substantial diversity across repositories, languages, agent architectures, and task types. The 102 instances span 35 distinct repositories across three programming languages: Python (73 instances, 71.6%), Go (24 instances, 23.5%), and JavaScript (5 instances, 4.9%). Trajectories are sourced from four structurally diverse agentic systems: SWE-agent (53), OpenHands (30), AutoCodeRover (10), and Live-SWE-agent (9). Tasks are predominantly bug fixing, with approximately 15% involving feature implementation. This diversity across repositories, languages, and agent architectures ensures that RootSE captures a broad range of real-world failure scenarios rather than artifacts of any single system or domain.

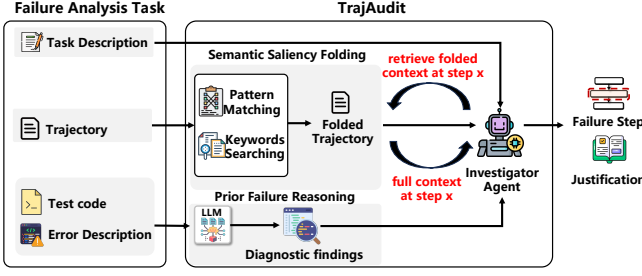


Fig. 5: The overall workflow of TrajAudit.

V. TRAJAUDIT METHODOLOGY

A. Overview

TrajAudit comprises an investigator agent and two supporting modules: a prior failure reasoning (PFR) module and a semantic saliency folding (SSF) module. As illustrated in Figure 5, SSF compresses failure-irrelevant observations into placeholders, reducing observational noise while preserving critical signals such as code patches and error indicators. PFR generates a preliminary diagnosis from the test code and error description, directing the investigator agent toward the most probable failure region to mitigate long-context degradation [40], [41]. Serving as a central hub, the investigator agent integrates both outputs and dynamically retrieves folded content on demand [28], enabling a focused top-down investigation without sacrificing access to the full trajectory. We detail each component in the following subsections.

B. Prior Failure Reasoning

The prior failure reasoning module aims to provide prior knowledge to guide the LLM toward the most relevant areas of the trajectory for diagnosis. This design is inspired by the debugging process of software engineers. Rather than analyzing an entire execution trajectory at once, engineers typically look at the error information (e.g., exception logs) to localize the potential failure region before expanding their scope of investigation. Although recent LLMs have substantially extended context windows, existing research demonstrates that long context reasoning still incurs significant performance degradation [42]. We therefore adopt a similar strategy to help the agent maintain focus when encountering long-horizon trajectories.

Specifically, this module accepts the failure information from task execution, including test code and error description, and prompts the LLM to identify the workflow phase most likely responsible for the failure along with the corresponding rationale, without constraining it to a predefined set of phases. This preliminary diagnostic serves as prior knowledge, directing the LLM’s attention toward the most probable failure region before it processes the full trajectory.

C. Semantic Saliency Folding

Compared to natural language reasoning or web browsing tasks, coding trajectories are much denser and more com-

plex, particularly within the observation steps that document code artifacts and execution states. In RootSE, for example, observation content accounts for over 74.9% of the total trajectory length. While certain observations contain critical failure signals (e.g., error logs or patch outputs), the majority constitute redundant data. For instance, creating a file often returns verbose outputs such as complete directory trees. All such tool responses are logged entirely within the observation entries of the trajectory. As these failure-irrelevant observations accumulate over long-horizon tasks, they introduce substantial noise that degrades the reasoning capabilities of diagnostic methods.

To mitigate this, we introduce the semantic saliency folding module, which identifies potential failure signals through pattern and keyword matching and replaces the remaining irrelevant observations with recoverable folding markers. We consider two types of observations as failure-relevant. The first encompasses the generated code patches, as they explicitly capture how the agent modifies the code and thus allow us to identify which exact code changes lead to the test failure. Second, inspired by run-time monitoring for traditional systems [43]–[47], we preserve the observations containing failure-indicative keywords, such as `traceback` and `exception`.

The semantic saliency folding is detailed in Algorithm 1. To begin with, we apply pattern matching to identify code patches. The widely adopted patch format in software projects is the code diff [48], which follows a unified structure: the patch header contains file metadata in the form of `--- a` and `+++ b`, representing the original and modified files respectively, while the hunk header follows the format `@@ -N,M +P,Q @@` to indicate the location of modifications within the code file. We encode these fixed structural patterns into regular expressions and apply them to identify patches within observations. Afterwards, we predefine a failure indicator dictionary covering the frequently occurring failure-indicative keywords, constructed through LLM generation and manual refinement. Any observation containing these keywords is flagged as a signal that the agentic system has likely encountered an anomalous state. Observations that lack both patch data and failure keywords are folded, while those matching either criterion are preserved. This folding process is reversible: when the compressed trajectory lacks sufficient evidence, the investigator agent can retrieve the original content of any folded observation through the inspection tool.

The resulting folded trajectory is then passed to the investigator agent for further analysis.

D. Investigator Agent

The investigator agent serves as a central hub that integrates the processed outputs from the above two modules, and dynamically probes the folded observations on demand to make the final diagnosis.

The investigator agent is prompted to first assess whether the current context is sufficient for failure diagnosis. If so, it directly outputs the identified failure step index along with the corresponding justification; otherwise, it iteratively invokes

Algorithm 1 Semantic Saliency Folding

Input: Trajectory T **Output:** Folded trajectory T^*

```
 $P \leftarrow$  compiled patch pattern  $\triangleright$  e.g., ---a, +++b, @@ -N, M +P, Q @@  
 $K \leftarrow$  {"exception", "fail", "error", "traceback", "invalid" ...}  
 $T^* \leftarrow []$   
for each entry  $c$  in  $T$  do  
   $d \leftarrow \text{parse}(c)$   
  if  $d["\text{observation}"] \not\sim P$  AND  $\nexists k \in K : k \in d["\text{observation}"]$  then  
     $d["\text{observation}"] \leftarrow$  "Folded, invoke API to view the full context"  
  end if  
  append  $\text{serialized}(d)$  to  $T^*$   
end for
```

local tools to inspect folded observations and progressively expands the available context until a final diagnosis can be produced.

To support this process, the agent is equipped with two interactive tools: one for retrieving the content of a folded observation at a specific step, and one for submitting the final diagnostic result.

VI. EXPERIMENTAL SETUP

We evaluate TrajAudit by answering the following research questions (RQs):

- **RQ1:** How effective is TrajAudit?
- **RQ2:** How does TrajAudit balance localization effectiveness and token cost compared with baselines?
- **RQ3:** How robust is TrajAudit across different backbones?
- **RQ4:** How does each component contribute to TrajAudit?

A. Dataset

We conduct experiments on RootSE, the dataset proposed in this paper (Section IV), comprising 102 instances collected from trajectories of representative agentic systems that failed to resolve real-world repository-level coding issues. Each instance is annotated with the earliest decisive error step and the corresponding diagnosis justification.

B. Baselines

We compare TrajAudit against five baselines described in Section III: three strategies proposed by Zhang et al. [20] (All-at-Once, Step-by-Step, Binary Search), TRAIL [21], and FAMAS [22]. We exclude AgenTracer [18] as its trained model parameters are not publicly available. To adapt these methods to RootSE, we embed the RootSE annotation guideline into every baseline’s prompt, ensuring all methods operate under the same definition of the earliest decisive error step. For TRAIL, we adopt a two-step pipeline. In the first step, we follow the original design by feeding the complete trajectory along with TRAIL’s error taxonomy to the LLM to identify all failed steps. In the second step, we provide all the identified failed steps together with the annotation guideline to the LLM, which selects the single earliest decisive error step. For FAMAS, we provide at least three trajectories per case for suspiciousness scoring, prioritizing successful trajectories when available. These trajectories are obtained from multiple

runs of the same agent system on the same task. For cases where no successful trajectory exists, we supplement with additional failed trajectories from different runs. The resulting suspiciousness rankings are then combined with the annotation guideline and fed to the LLM to produce the final prediction. For All-at-Once, Step-by-Step, and Binary Search, we incorporate the annotation guidelines into the prompts. Cases where the trajectory exceeds the LLM’s context limit are treated as diagnosis failures.

C. Evaluation Settings

Inspired by the evaluation protocol of Who&When, we conduct experiments under two settings: **with reference** and **without reference**, depending on whether a reference patch for the current task is provided. Unlike Who&When, where tasks have deterministic answers, repository-level coding tasks support multiple valid solutions, making a single definitive solution patch infeasible. We therefore provide a reference patch as one representative solution.

These two settings evaluate failure diagnosis methods under different real-world scenarios. The with-reference setting reflects the common development scenario where agentic systems are evaluated on benchmark tasks with known reference solutions, allowing diagnosis methods to leverage reference information when identifying failure causes. In contrast, the without-reference setting reflects practical deployment scenarios where no reference solution is available and diagnosis must rely solely on execution logs.

D. Metrics

We evaluate all methods using three metrics introduced in Section IV-A. To validate the reliability of the LLM-as-a-judge approach used for Justification Accuracy, we randomly sampled 30 instances per method (180 instances in total) and had two annotators independently assess whether each predicted justification is semantically equivalent to the ground truth. The two annotators reached an initial agreement rate of 97.2%, and all remaining disagreements were resolved through discussion to produce a consensus label. The LLM-as-a-judge achieved an agreement rate of 93.3% with the consensus labels, indicating sufficient reliability as an automated metric. For short, we denote step-level accuracy under tolerance k as $\text{Tol}@k$. Unless otherwise specified, we use $\text{Tol}@0$ under the without-reference setting as the primary metric.

E. Implementation Details

We employ Claude Sonnet 4.5¹ as the default backbone LLM. To reduce randomness and improve reproducibility, the temperature is set to 0.

VII. EVALUATION

A. RQ1. How effective is TrajAudit?

We comprehensively evaluate the effectiveness of TrajAudit against state-of-the-art baselines across four dimensions: exact step-level accuracy, tolerated step-level accuracy, performance across varying trajectory lengths, and justification accuracy.

¹claude-sonnet-4-5-20250929

1) *Exact Step-level Accuracy*: As shown in Table II, TRAIL and FAMAS achieve relatively low exact step-level accuracy on RootSE, despite their competitive performance on general-purpose agentic tasks such as web browsing and file understanding. This suggests that these methods do not generalize well to repository-level coding task trajectories. In contrast, TrajAudit achieves the highest exact step-level accuracy in both settings. It outperforms the best-performing baseline TRAIL by 10.8 percentage points (49.0% vs. 38.2%) with reference and by 21.6 percentage points (52.0% vs. 30.4%) without reference.

TABLE II: Exact step-level accuracy (%) on RootSE.

Category	Method	w/ Reference	w/o Reference
LLM-based	All-at-Once	27.5	24.5
	Step-by-Step	23.5	22.6
	Binary Search	18.6	11.8
	TRAIL	<u>38.2</u>	<u>30.4</u>
Spectrum-based	FAMAS	25.5	<u>30.4</u>
Agent-based	TrajAudit	49.0	52.0

[†] Cases exceeding the LLM context limit are treated as diagnosis failures (All-at-Once: 3, Binary Search: 2, TRAIL: 1).

To confirm statistical significance, we apply exact McNemar’s test on per-instance paired results under the without-reference setting, where b and c denote the number of instances correctly diagnosed only by TrajAudit and only by the baseline, respectively. As shown in Table III, TrajAudit significantly outperforms all five baselines ($p < 0.001$ in each case), with substantially larger b than c in every comparison. These results indicate that TrajAudit’s advantage is consistent across instances and is not attributable to chance.

We further observe that the relative effectiveness of the with- and without-reference settings varies across methods. For methods that process the full trajectory in a single pass (All-at-Once, Step-by-Step, Binary Search, and TRAIL), the with-reference setting consistently yields higher accuracy. We attribute this to the reference patch providing an anchor for these methods: by identifying the intended code changes, the LLM can narrow its focus to relevant trajectory regions despite the long and noisy context. In contrast, TrajAudit and FAMAS perform better in the without-reference setting. Both methods employ mechanisms that already provide focused step-level views of the trajectory, namely active on-demand

TABLE III: McNemar’s test results comparing TrajAudit against each baseline (Tol@0, without reference). b = instances correctly diagnosed only by TrajAudit; c = instances correctly diagnosed only by the baseline.

Category	Baseline	b	c	p -value
LLM-based	All-at-Once	34	6	< 0.001
	Step-by-Step	42	12	< 0.001
	Binary-Search	44	3	< 0.001
	TRAIL	29	7	< 0.001
Spectrum-based	FAMAS	28	6	< 0.001

TABLE IV: Step-level accuracy under different tolerances (without-reference setting).

Category	Method	Tol@0	Tol@1	Tol@2	Tol@3
LLM-based	All-at-Once	24.5	41.2	50.0	<u>52.0</u>
	Step-by-Step	22.6	34.3	41.2	44.1
	Binary Search	11.8	27.5	34.3	39.2
	TRAIL	<u>30.4</u>	42.2	49.0	50.0
Spectrum-based	FAMAS	<u>30.4</u>	<u>44.1</u>	<u>51.0</u>	<u>52.0</u>
Agent-based	TrajAudit	52.0	58.8	62.8	65.7

inspection and suspiciousness-based clustering respectively, reducing their reliance on external anchoring. In this case, the reference patch may instead introduce bias by directing the LLM toward patch-related code changes, causing it to overlook earlier decisive errors in planning or exploration phases.

2) *Tolerated Step-level Accuracy*: In many practical scenarios, pinpointing the exact failure step is not always necessary; localizing the failure within a narrow range of candidate steps is often sufficient for downstream debugging and correction. Table IV reports step-level accuracy under different tolerance windows in the without-reference setting. As the tolerance increases, all methods improve, yet TrajAudit consistently achieves the highest accuracy across all tolerance levels. At ± 3 , TrajAudit reaches 65.7%, outperforming TRAIL and FAMAS by 15.7 and 13.7 percentage points, respectively. This consistent gap suggests that TrajAudit not only localizes failure steps more precisely, but also maintains its advantage under relaxed evaluation criteria.

3) *Performance across Varying Trajectory Lengths*: Figure 6 reports exact step-level accuracy across different trajectory-length buckets in the without-reference setting. TrajAudit achieves the highest accuracy in every bucket, indicating that its advantage is not limited to trajectories of a specific length. For example, TrajAudit reaches 46.7% accuracy on trajectories with 53–70 steps and 55.0% accuracy on trajectories with at least 71 steps, while the best-performing baseline achieves 30.0% and 35.0% in the same buckets, respectively.

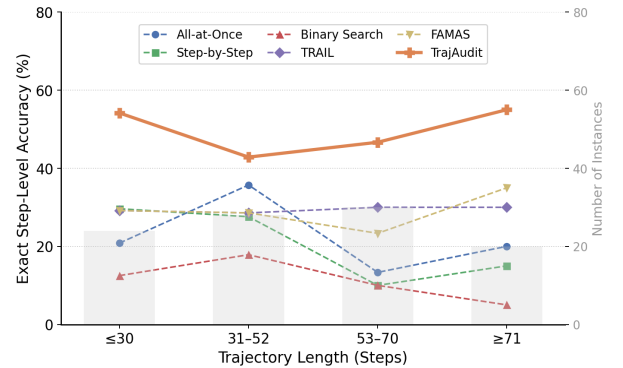


Fig. 6: Exact step-level accuracy across varying trajectory length on RootSE.

4) *Justification Accuracy*: Beyond step-level localization accuracy, we further evaluate whether each method produces

TABLE V: Justification accuracy (%) between predicted and ground-truth justifications in the without-reference setting.

	All-at-Once	Step-by-Step	TRAIL	FAMAS	TrajAudit
Accuracy (%)	<u>42.4</u>	14.7	40.6	28.4	52.9

a justification that correctly identifies the annotated failure cause. Table V reports the justification accuracy of each method, assessed by Claude Sonnet 4.5 as the LLM evaluator using the same evaluation rubric for all methods. Binary Search is excluded from this comparison, as it outputs only a region label (upper or lower half) at each iteration and produces no diagnostic justification. TrajAudit achieves the highest justification accuracy of 52.9%, outperforming the best baseline (All-at-Once, 42.4%) by 10.5 percentage points, while Step-by-Step yields only 14.7%, suggesting that its fragmented per-step analysis may struggle to capture the broader trajectory context required for accurate failure attribution.

B. RQ2. How does TrajAudit balance localization effectiveness and token cost compared with baselines?

Figure 7 compares all methods under the without-reference setting in terms of exact step-level accuracy and average API token consumption per case, where the latter is computed as total tokens consumed divided by the 102 benchmark cases. TrajAudit achieves the highest accuracy of 52.0% at an average cost of 138K tokens per case. It strictly dominates Binary Search (169K, 11.8%) and Step-by-Step (306K, 22.6%), achieving higher accuracy while consuming fewer tokens. Compared with the lower-cost baseline FAMAS (47K, 30.4%), TrajAudit requires more tokens but improves accuracy by 21.6 percentage points. The remaining baselines are dominated by either FAMAS or TrajAudit. Together with FAMAS, TrajAudit lies on the observed Pareto frontier among the evaluated methods, i.e., neither method is dominated by another evaluated method in both accuracy and token cost. This places TrajAudit at the high-accuracy end of the frontier and indicates a favorable balance between localization effectiveness and computational cost when accuracy is prioritized.

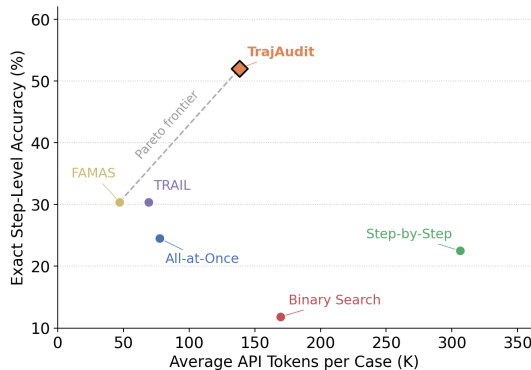


Fig. 7: Effectiveness–cost trade-off under the without-reference setting (Tolerance=0). The dashed line connects the observed non-dominated methods for visualization only.

TABLE VI: TrajAudit performance with different backbone LLMs on RootSE.

Backbone LLM	Tol@0	Tol@1	Tol@2	Tol@3	Justif.
✦ GEMINI 3.1 PRO	55.9	65.7	67.7	70.6	45.9
✧ CLAUDE SONNET 4.5	52.0	58.8	62.8	65.7	52.9
⊗ GPT-5.2	45.1	54.9	59.8	63.7	52.0
⚡ DEEPSEEK V4 PRO	45.1	57.8	61.8	64.7	42.6
⚡ QWEN3-235B	40.2	52.9	58.8	61.8	32.4

C. RQ3. How robust is TrajAudit across different backbones?

Table VI reports TrajAudit’s performance with five different backbone LLMs. Across all backbones, TrajAudit achieves 40.2–55.9% Tol@0 and 61.8–70.6% Tol@3. Even with the weakest evaluated backbone, TrajAudit achieves a higher Tol@0 than the best-performing baseline (TRAIL, 30.4%) reported in RQ1 under the same without-reference setting, supporting that its improvement is not specific to a single backbone choice. However, the 15.7 percentage-point Tol@0 gap and the 20.5 percentage-point justification gap between the strongest and weakest backbones suggest that backbone choice still affects both localization and explanation quality.

The results further reveal that localization accuracy and justification accuracy are not perfectly aligned. Gemini 3.1 Pro achieves the best localization performance (55.9% Tol@0, 70.6% Tol@3), yet its justification accuracy (45.9%) is lower than that of Claude Sonnet 4.5 and GPT-5.2. Conversely, Claude Sonnet 4.5 obtains the highest justification accuracy of 52.9%, despite ranking second in localization accuracy. These results suggest that identifying the faulty step and articulating a semantically accurate explanation require related but distinct capabilities. Overall, TrajAudit remains applicable across diverse backbone LLMs, while backbone choice continues to influence both localization accuracy and explanation quality.

D. RQ4. How does each component contribute to TrajAudit?

TABLE VII: Ablation study of TrajAudit components in the without-reference setting.

SSF	PFR	Exact Acc. (%)	Just. Acc. (%)	Avg. Tokens (K)
✓	✓	52.0	52.9	138.2
✓	×	34.3 (↓17.7)	38.0 (↓14.9)	125.5 (↓12.7)
×	✓	38.2 (↓13.8)	36.0 (↓16.9)	86.2 (↓52.0)

Numbers in parentheses denote absolute changes from the full model; accuracy in percentage points, tokens in K.

SSF: Semantic Saliency Folding; PFR: Prior Failure Reasoning.

Table VII presents the ablation results for each component of TrajAudit in the without-reference setting. Removing PFR causes the largest drop in exact step-level accuracy, decreasing it by 17.7% while reducing token consumption by only 12.7K tokens per case, suggesting that PFR provides a strong localization prior with relatively small token overhead. Removing SSF reduces token consumption by 52.0K tokens per case but also decreases exact accuracy by 13.8% and justification accuracy by 16.9%. Although SSF folds the

majority of trajectory steps upfront, TrajAudit’s iterative multi-step localization process accumulates tokens across successive rounds of on-demand expansion calls, resulting in higher overall consumption than the variant without SSF. Overall, this indicates that SSF’s token overhead is accompanied by substantial gains in both localization and justification quality.

To further understand SSF’s behavior, we analyze its folding operations across all 102 benchmark cases. On average, SSF folds the observations of 94.6% of trajectory steps per case, retaining the full observations of only 5.4% of steps, primarily those containing patch content or failure-related signals such as exceptions and tracebacks. Among the folded steps, the agent subsequently expands only 20.2% on demand via targeted tool calls, leaving over 75% of all trajectory steps never expanded in full. These results suggest that SSF enables a focused investigation strategy: it aggressively abstracts low-saliency steps while allowing the agent to selectively retrieve specific context when needed.

VIII. TRAJAUDIT ERROR ANALYSIS

To understand the limitations of TrajAudit, we analyze the cases where TrajAudit fails to localize the annotated earliest decisive error step. We find that the localization accuracy correlates with the position of the annotated error step. When the error occurs within the first 20% of steps, accuracy drops to 19%–26% across backbones, whereas errors in the middle portion (35%–65%) achieve substantially higher accuracy of 54%–57%. Early errors are difficult to localize because they leave no observable transition from correct to incorrect behavior [49]. The entire subsequent trajectory executes on a flawed basis, with no clear turning point to identify. When the error occurs in the middle of the trajectory, by contrast, TrajAudit can draw on the contrast between the preceding correct steps and the subsequent erroneous ones to localize the mistake more reliably.

To better understand these failures, we identify two recurring patterns among TrajAudit’s mislocalized cases. (1) Some agents commit to a flawed problem framing in the initial planning stage before making any concrete code change. TrajAudit is more reliable when erroneous reasoning is reflected in observable code or tool-use evidence, but less effective when the decisive error remains implicit in an early plan. Consequently, it often selects a later step where the flawed plan first becomes externally observable. (2) Even when the root cause appears in code early in the trajectory, its effects may accumulate silently over many subsequent steps, leading TrajAudit to identify the step where the effects of the error become most evident rather than where the mistake was first introduced. Both patterns produce the same directional bias: among mislocalized predictions, 81% are later than the annotated step, with an average delay of 14.5 steps, as illustrated in Figure 8.

Successful localizations typically contain a distinct transition from correct exploration to one identifiable erroneous decision that directly causes the task failure. These findings suggest that future work should strengthen reasoning-level

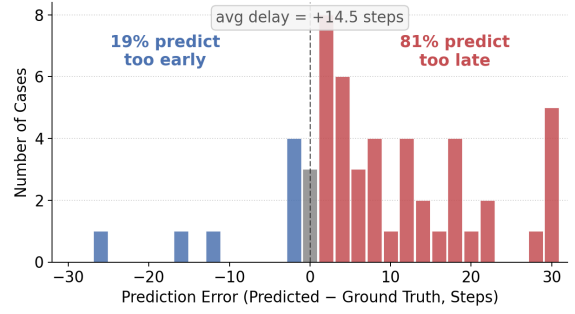


Fig. 8: Distribution of prediction offsets (predicted step - ground-truth step) among mislocalized cases.

localization, especially for early planning errors whose consequences only become observable much later in the trajectory.

IX. THREATS TO VALIDITY

Internal validity. The main internal threats concern annotation reliability and the nondeterminism of LLM-based methods. We mitigate annotation subjectivity by using a unified guideline, independent annotation by two annotators, and arbitration by a third validator. For LLM-based methods and evaluators, we set the temperature to 0 across all experiments to reduce sampling variance.

External validity. The main external threat concerns generalizability. First, RootSE contains 102 instances filtered from 500 trajectories, excluding cases primarily attributable to external factors such as ambiguous task descriptions. This filtering improves annotation reliability and aligns with our focus on agent reasoning and action failures, but may bias the dataset away from failures caused by task ambiguity or other external conditions. Second, RootSE currently covers bug fixing and feature implementation tasks, and does not include other repository-level coding tasks such as test case generation [50], which may limit the generalizability of our findings. At the same time, RootSE is comparable in scale to related benchmarks, including Who&When [20] with 184 instances and the study by Bouzenia et al. [19] with 120 trajectories. Future work will extend the benchmark to broader task types and failure sources.

X. CONCLUSION

In this paper, we present TrajAudit, an automated failure diagnosis framework for repository-level coding agentic trajectories. By combining semantic saliency folding with prior failure reasoning, TrajAudit filters irrelevant context and guides an investigator agent toward likely causal steps in noisy trajectories. We also construct RootSE, a benchmark of 102 failed repository-level coding task trajectories with step-level earliest-error annotations. Experimental results demonstrate that TrajAudit consistently outperforms all baselines on RootSE, demonstrating its effectiveness.

DATA AVAILABILITY

The complete RootSE benchmark, the TrajAudit implementation, and all artifacts required to reproduce the reported results are publicly available at: <https://github.com/LogAnalysisTech/TrajAudit>.

REFERENCES

- [1] S. V. Albrecht and P. Stone, "Autonomous agents modelling other agents: A comprehensive survey and open problems," *Artificial Intelligence*, vol. 258, pp. 66–95, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0004370218300249>
- [2] S. Franklin and A. Graesser, "Is it an agent, or just a program? a taxonomy for autonomous agents," in *Proceedings of the Workshop on Intelligent Agents III, Agent Theories, Architectures, and Languages*, ser. ECAI '96. Berlin, Heidelberg: Springer-Verlag, 1996, p. 21–35.
- [3] J. Luo, W. Zhang, Y. Yuan, Y. Zhao, J. Yang, Y. Gu, B. Wu, B. Chen, Z. Qiao, Q. Long *et al.*, "Large language model agent: A survey on methodology, applications and challenges," *arXiv preprint arXiv:2503.21460*, 2025.
- [4] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VtmBAGCN7o>
- [5] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 1592–1604. [Online]. Available: <https://doi.org/10.1145/3650212.3680384>
- [6] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, "Openhands: An open platform for ai software developers as generalist agents," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:271404773>
- [7] L. Hu, G. Chen, X. Shang, S. Cheng, B. Wu, L. LiGangyang, X. Zhu, W. Zhang, and N. Yu, "Compileagent: Automated real-world repo-level compilation with tool-integrated llm-based agent system," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2025, Vienna, Austria, July 27 - August 1, 2025, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Association for Computational Linguistics, 2025, pp. 2078–2091. [Online]. Available: <https://doi.org/10.18653/v1/2025.acl-long.103>
- [8] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 15174–15186. [Online]. Available: <https://aclanthology.org/2024.acl-long.810/>
- [9] S. Han, Q. Zhang, W. Jin, and Z. Xu, "Llm multi-agent systems: Challenges and open problems," *arXiv preprint arXiv:2402.03578*, 2024.
- [10] W. Epperson, G. Bansal, V. C. Dibia, A. Fournay, J. Gerrits, E. E. Zhu, and S. Amershi, "Interactive debugging and steering of multi-agent ai systems," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI '25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3706598.3713581>
- [11] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 1482–1494. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00129>
- [12] T. Liu, C. Xu, and J. McAuley, "Repubench: Benchmarking repository-level code auto-completion systems," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 47832–47850.
- [13] C. Parnin and A. Orso, "Are automated debugging techniques actually helping programmers?" in *Proceedings of the 2011 International Symposium on Software Testing and Analysis*, ser. ISSTA '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 199–209. [Online]. Available: <https://doi.org/10.1145/2001420.2001445>
- [14] A. Barrak, "Traceability and accountability in role-specialized multi-agent llm pipelines," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, 2025, pp. 315–322.
- [15] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, Dec. 2024. [Online]. Available: <https://doi.org/10.1145/3695988>
- [16] R. Lu, Y. Li, and Y. Huo, "Exploring autonomous agents: A closer look at why they fail when completing tasks," 11 2025, pp. 3856–3860.
- [17] M. Z. Pan, M. Cemri, L. A. Agrawal, S. Yang, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, K. Ramchandran, D. Klein, J. E. Gonzalez, M. Zaharia, and I. Stoica, "Why do multiagent systems fail?" in *ICLR 2025 Workshop on Building Trust in Language Models and Applications*, 2025. [Online]. Available: <https://openreview.net/forum?id=wM521FqPv1>
- [18] G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, and S. YAN, "Agentracer: Who is inducing failure in the LLM agentic systems?" in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=i05DseqvUD>
- [19] I. Bouzenia and M. Pradel, "Understanding software engineering agents: A study of thought-action-result trajectories," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Press, 2025, p. 2846–2857. [Online]. Available: <https://doi.org/10.1109/ASE63991.2025.00234>
- [20] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. ICML'25. JMLR.org, 2025.
- [21] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian, "Trail: Trace reasoning and agentic issue localization," 2025. [Online]. Available: <https://arxiv.org/abs/2505.08638>
- [22] Y. Ge, L. Xie, Z. Li, Y. Pei, and T. Zhang, "Who is introducing the failure? automatically attributing failures of multi-agent systems via spectrum analysis," *arXiv preprint arXiv:2509.13782*, 2025.
- [23] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. Chi, N. Schärli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.
- [24] S. Chaudhury, P. Das, S. Swaminathan, G. Kollias, E. Nelson, K. Pahwa, T. Pedapati, I. Melnyk, and M. Riemer, "EpMAN: Episodic memory Attention for generalizing to longer contexts," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 11696–11708. [Online]. Available: <https://aclanthology.org/2025.acl-long.574/>
- [25] Y. Tian and T. Zhang, "Selective prompt anchoring for code generation," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. ICML'25. JMLR.org, 2025.
- [26] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. [Online]. Available: <https://aclanthology.org/2024.tacl-1.9/>
- [27] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "Toolllm: Facilitating large language models to master 16000+ real-world apis," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=dHng200Jjr>
- [28] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in neural information processing systems*, vol. 36, pp. 68539–68551, 2023.

- [29] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VTF8yNQm66>
- [30] G. Mialon, C. Fourrier, T. Wolf, Y. LeCun, and T. Scialom, "Gaia: a benchmark for general ai assistants," in *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., vol. 2024, 2024, pp. 9025–9049. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/25ae35b5b1738d80f1f03a8713e405ec-Paper-Conference.pdf
- [31] X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane *et al.*, "Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?" *arXiv preprint arXiv:2509.16941*, 2025.
- [32] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "Swe-agent: agent-computer interfaces enable automated software engineering," in *Proceedings of the 38th International Conference on Neural Information Processing Systems*, ser. NIPS '24. Red Hook, NY, USA: Curran Associates Inc., 2024.
- [33] C. S. Xia, Z. Wang, Y. Yang, Y. Wei, and L. Zhang, "Live-swe-agent: Can software engineering agents self-evolve on the fly?" *arXiv preprint arXiv:2511.13646*, 2025.
- [34] I. Badertdinov, A. Golubev, M. Nekrashevich, A. Shevtsov, S. Karasik, A. Andriushchenko, M. Trofimova, D. Litvintseva, and B. Yangel, "SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. [Online]. Available: <https://openreview.net/forum?id=nMpJoVmRyl>
- [35] M. Trofimova, A. Shevtsov, B. Ibragim, K. Pyaev, S. Karasik, and A. Golubev, "Openhands trajectories with qwen3-coder-480b-a35b-instruct," *Nebius blog*, 2025.
- [36] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *biometrics*, pp. 159–174, 1977.
- [37] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and psychological measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [38] B. Kitchenham, O. Pearl Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, "Systematic literature reviews in software engineering - a systematic literature review," *Inf. Softw. Technol.*, vol. 51, no. 1, p. 7–15, Jan. 2009. [Online]. Available: <https://doi.org/10.1016/j.infsof.2008.09.009>
- [39] Q. Zhang, C. Fang, Y. Ma, W. Sun, and Z. Chen, "A survey of learning-based automated program repair," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 2, Dec. 2023. [Online]. Available: <https://doi.org/10.1145/3631974>
- [40] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [41] M. Weiser, "Program slicing," in *Proceedings of the 5th International Conference on Software Engineering*, ser. ICSE '81. IEEE Press, 1981, p. 439–449.
- [42] Y. Zhou, H. Liu, Z. Chen, Y. Tian, and B. Chen, "Gsm- ∞ : how do your llms behave over infinitely increasing reasoning complexity and context length?" in *Proceedings of the 42nd International Conference on Machine Learning*, ser. ICML'25. JMLR.org, 2025.
- [43] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, p. 507–525, May 2015. [Online]. Available: <https://doi.org/10.1109/TSE.2014.2372785>
- [44] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 1285–1298. [Online]. Available: <https://doi.org/10.1145/3133956.3134015>
- [45] H. Guo, S. Yuan, and X. Wu, "Logbert: Log anomaly detection via bert," in *2021 International Joint Conference on Neural Networks (IJCNN)*, 2021, pp. 1–8.
- [46] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*, 2016, pp. 207–218.
- [47] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Machine Learning with Applications*, vol. 12, p. 100470, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666827023000233>
- [48] E. W. Myers, "Ano(nd) difference algorithm and its variations," *Algorithmica*, vol. 1, no. 1–4, p. 251–266, Nov. 1986. [Online]. Available: <https://doi.org/10.1007/BF01840446>
- [49] A. Zeller, "Yesterday, my program worked. today, it does not. why?" in *Proceedings of the 7th European Software Engineering Conference Held Jointly with the 7th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. ESEC/FSE-7. Berlin, Heidelberg: Springer-Verlag, 1999, p. 253–267.
- [50] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.